A Network Arena for Benchmarking AI Agents on Network Troubleshooting

ZHIHAO WANG, University of Electronic Science and Technology of China (UESTC), China
ALESSANDRO CORNACCHIA, KAUST, Saudi Arabia
ALESSIO SACCO, Politecnico di Torino, Italy
FRANCO GALANTE, Politecnico di Torino, Italy
MARCO CANINI, KAUST, Saudi Arabia
DINGDE JIANG, University of Electronic Science and Technology of China (UESTC), China

Agentic systems, powered by Large Language Models (LLMs), assist network engineers with network configuration synthesis and network troubleshooting tasks. For network troubleshooting, progress is hindered by the absence of standardized and accessible benchmarks for evaluating LLM agents in dynamic network settings at low operational effort. We present NIKA, the largest public benchmark to date for LLM-driven network incident diagnosis and troubleshooting. NIKA targets both domain experts and especially AI researchers alike, providing zero-effort replay of real-world network scenarios, and establishing well-defined agent-network interfaces for quick agent prototyping. NIKA comprises hundreds of curated network incidents, spanning five network scenarios, from data centers to ISP networks, and covers 54 representative network issues. Lastly, NIKA is modular and extensible by design, offering APIs to facilitate the integration of new network scenarios and failure cases. We evaluate state-of-the-art LLM agents on NIKA and find that while larger models succeed more often in detecting network issues, they still struggle to localize faults and identify root causes. NIKA is open-source and available to the community: https://github.com/sands-lab/nika.

1 INTRODUCTION

Artificial Intelligence (AI) is shifting from human-oriented chat applications to autonomous agentic systems with decision-making capabilities. Google has projected a 1 trillion USD market value for agentic AI by 2040 [16], with Large Language Models (LLMs) at the core of this transformation. This evolution has accelerated the adoption of LLMs in domains requiring dynamic, context-aware operations, such as networking [6, 21, 41, 44, 62–64, 67, 68, 71] and systems [15, 18, 25, 55, 57, 69, 78, 80]. Network troubleshooting is a compelling target: given a network incident, engineers undertake mechanical yet cumbersome steps to identify and remediate issues, from probing the network to explain packet drops, to refining detection logic across heterogeneous telemetry and diagnosis tools [8, 23]. The process is manual, slow, and prone to errors, requiring expert operators to reason across multiple dimensions. Recent work shows that cloud operators and telco vendors have already deployed AI troubleshooting agents for production networks [3, 63, 64, 68], leveraging LLMs to parse textual data from monitoring tools and assist operators in failure detection and localization.

Despite the broad consensus, our community still lacks the benchmarks and platforms needed to explore and compare agent designs in a principled way. For example, existing systems such as NetAssistant [64], BIAN [63], and Confucius [68] are evaluated in closed settings with no publicly available datasets and limited disclosure of evaluation scenarios. NetConfEval [62] instead provides a public benchmark but focuses on static configuration synthesis, making it impossible for live network troubleshooting or studying how well agents sequence decision-making. Additionally, it cannot capture important dimensions that extend beyond accuracy, such as time to resolution,

Authors' Contact Information: Zhihao Wang, University of Electronic Science and Technology of China (UESTC), China; Alessandro Cornacchia, KAUST, Saudi Arabia; Alessio Sacco, Politecnico di Torino, Italy; Franco Galante, Politecnico di Torino, Italy; Marco Canini, KAUST, Saudi Arabia; Dingde Jiang, University of Electronic Science and Technology of China (UESTC), China.

reasoning quality, or the impact on network performance during diagnosis, which are equally critical in troubleshooting tasks.

We can observe two gaps in the existing solutions. First, designers of AI agents need to evaluate a rich design space that includes, among others: *agent structure*, *e.g.,* monolithic versus specialized sub-agents [40, 50, 87]; *prompting strategies*, *e.g.,* how to structure the agent's reasoning process [34, 43, 66, 70]; *tool interfaces*, *e.g.,* granularity and abstraction of external APIs [10, 51]; *state management*, *e.g.,* which historical data to keep in the context at each reasoning step [38, 75]. Because AI programmers do not necessarily possess domain knowledge, they expect a ready-to-use benchmark to evaluate the impact of their design choices on agent behaviors at low operational effort. Second, experimenting with network troubleshooting scenarios requires a controlled network environment in which to connect AI agents and reproduce failures without disrupting normal user traffic and network operations. Production networks are unsuitable due to safety, privacy, and reproducibility constraints. Existing emulators provide controllable networks, but fall short of offering: *(i)* streamlined reproducibility of given incident scenarios; *(ii)* a unified interface to connect AI agents with the network environment; *(iii)* scoring mechanisms for task objectives (*e.g.,* localization accuracy) and reasoning-level evaluations.

To fill these gaps, we present **NIKA**, a **N**etwork **I**ncidents benchmar**K**ing FrameworK for **AI** Agents. NIKA is a unified platform that can offer: (1) a benchmark suite of curated network incidents that covers 54 realistic network issues, ranging from link and host failures to resource contention, and includes five network scenarios, four of which can be instantiated at different topology sizes, spanning campus and data center networks. By combining these dimensions, the benchmark yields 640 distinct troubleshooting incidents for evaluating AI agents. The benchmark can be further extended by randomizing failure locations and composing multiple issues within a single incident. (2) A modular plug-and-play orchestration platform that connects AI agents with the network environment, enabling real-time troubleshooting in realistic conditions, and providing a human-facing interface to monitor agent performance.

NIKA orchestrates behind the scenes the components required for traffic generation, failure injection, integration with specialized telemetry instrumentation and diagnosis tools, observability, performance evaluation, and reasoning trajectory analysis, leaving AI programmers to focus only on agent design and evaluation. Through the Model Context Protocol (MCP) [7], the framework exposes more than 30 monitoring and troubleshooting tools to third-party agents, ranging from sketches and INT [49] to SDN controller APIs and switch CLIs. Thus, we used NIKA to evaluate three popular LLMs, namely GPT-OSS, GPT-5, and GPT-5-mini, and analyzed how these models behave when troubleshooting various network failures. We have found that, for example, although larger models are more successful in detecting network issues, they still struggle to localize faults and identify root causes.

Contributions. In summary, this paper makes the following contributions:

- we formalize the problem of benchmarking AI agents for network troubleshooting, identifying key requirements and challenges;
- we design a structured methodology to benchmark AI agents in this context and implement a modular platform that orchestrates the end-to-end evaluation workflow and outputs detailed observability and performance reports;
- we evaluate three state-of-the-art LLMs to validate the benchmark rigor, summarizing the problems detected with NIKA and share our experience in dealing with them;
- we release the framework [84] and disclose a large public dataset of AI agents' behavior for network troubleshooting, with more than 900 reasoning traces.

2 BACKGROUND & MOTIVATION

Network troubleshooting. Network incidents are inevitable with the scale and complexity of current production networks. Hardware failures, misconfigurations, and software bugs, are common causes of network incidents [45, 63, 86]. Because incidents lead to degraded performance and service outages, operators must promptly resolve them and minimize the impact on customers. Over decades of networking research, a humongous amount of network monitoring and diagnosis tools have been developed to assist operators in troubleshooting incidents in their networks [8, 20, 32, 46, 61]. In addition to basic networking tools (*e.g.*, `ping`, `traceroute`, etc.), operators leverage advanced diagnostics frameworks such as PingMesh [27], 007 [8], deTector [52] and others to monitor network health, detect anomalies, and diagnose root causes automatically [11, 23, 24, 28, 30, 59, 65]. Despite these advancements, *manual* intervention is still necessary in the presence of never-seen-before anomalies, or anomalies that sit in the tail percentiles and evade detection. To date, large providers report more than 200 incidents per week being resolved manually in production networks [63], or remain unsolved [45].

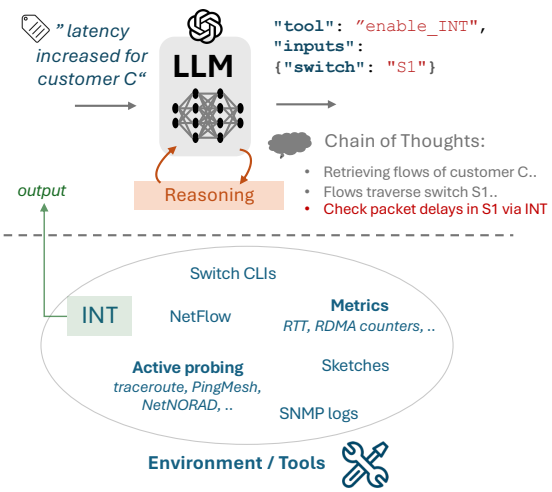


Fig. 1. Network troubleshooting with an LLM agent.

Manual incident management outpaces human capacity. In a typical troubleshooting workflow, network engineers start receiving tickets containing an incident description. These are high-level descriptions of symptoms reported by customers or automated monitoring systems, *e.g.*, "increased latency between region A and B", "connectivity loss for service S", etc. Subsequently, engineers scrutinize multiple alerts generated by their anomaly detection tools and decide which telemetry to inspect in depth. Examples include logs from network devices, flow-level telemetry (such as NetFlow [39] or sFlow [54]), and information from data plane telemetry systems (such as INT [11, 49, 83] or sketches [36, 47, 76]). Because alerts cardinality scales with the network size and number of monitoring tools, it is common to see hundreds of alerts for a single incident [33, 82]. Engineers must decide which telemetry to prioritize, reason across multiple signals, and perform manual correlation and causation analysis. For example, engineers must perform active probing on selected paths or refine the detection logic, *e.g.*, by requesting fine-grained queue-length data to zoom into an ongoing incident [26], enable debug-level diagnostics, etc., until the root cause is identified and remediated. The process is iterative and time-consuming, admittedly stretching the available human capacity in cloud networks [63].

State-of-the-art with AI agents. Agentic systems replace or augment human operators with LLM-powered agents during the diagnostic workflow [63, 64, 68]. An LLM-powered *AI agent* combines one or more LLMs with the ability to invoke external *tools* – function calls that let the models interact with the outside world. Fig. 1 illustrates the mechanism for a ReAct-style [77] reasoning agent. Given a high-level symptom description such as "latency has increased for customer C's traffic", the model first performs reasoning steps to map symptoms to potential causes and required telemetry. [1] Based on the reasoning steps, the model generates a tool call, specifying the

[1] A *system* prompt instructs the model with the available tools and the agent role (omitted in Fig. 1 for brevity).

function to invoke (`enable_INT`) and its parameters (`switch: S1`). The tool executes in the network environment and returns an output, which the model incorporates into subsequent reasoning. [2]

Large-scale production deployments already validate that LLM-based agents can effectively assist network troubleshooting. BiAn [63] achieved 95.5% accuracy in localizing faulty devices across 357 incidents at Alibaba Cloud. Meta's Confucius [68] generalizes this line of work to encompass network management at a large scale. NetAssistant [64] is a copilot for on-call engineers at hyperscaler networks.

Evaluation gap. However, as the number of relevant applications grows, the establishment of a unified evaluation framework and benchmark has become a critical challenge for the community, which needs to evaluate agent designs consistently and fairly. Currently, scientific research in this area is fragmented and difficult to compare.

To improve performance and reduce costs, programmers of AI agents must evaluate a rich design space spanning agent architectures, prompting strategies, state management, and tool interfaces. Yet, no universally accepted benchmark targets network troubleshooting to date, thus creating an evaluation gap for systematic exploration of the problem space. NetConfEval [62] focuses on configuration synthesis, without closed-loop interaction with a network environment and its dynamically evolving state. Systems such as BiAn [63] and Confucius [68] are evaluated in closed settings with no publicly available datasets, and limited disclosure of the network scenarios. It is currently unclear how to advance state-of-the-art, especially for players outside cloud providers or ISPs. For example, it would be desirable to assess whether BiAn's design choices (*e.g.,* tool interfaces, prompting strategies, agent structure) generalize to heterogeneous network types, and, if so, how that would impact performance.

Network emulation platforms [12, 53, 58, 60] provide controlled environments and can simulate network incidents. However, a curated problem set of realistic network incidents is missing. Even when available from operators, developers would still shoulder the effort of reproducing the incidents with high fidelity, *e.g.,* trigger failure events in the presence of given traffic patterns, application-level states and network configurations – not a trivial task without domain expertise and a time-consuming process with high chances of errors when using low-level emulation APIs. In addition, these platforms also lack interfaces for agent-network interaction, requiring programmers to build tool-specific adapters to expose the network environment to AI agents. Lastly, there are no standardized evaluation schemes to compare agents' solutions against predefined success criteria, which typically depend on the objective of the troubleshooting task (Sec. 3.2) at hand. These limitations either lead programmers of AI agents to build custom scenarios for quick prototyping, or dissuade them from experimenting with this research area altogether.

3 NIKA

We address the evaluation gap with `NIKA`. In this section, we discuss our methodology and its key components.

3.1 Framework overview

`NIKA` is a framework to benchmark troubleshooting AI agents on curated network incidents. Its users are developers of AI agents wishing to evaluate a given design. As illustrated in Figure 2, `NIKA` separates concerns between AI agent development, network environment management, and

[2] More complex agent architectures explicitly structure the reasoning process as a graph of API calls to LLMs, each with predefined prompt templates and input-output dependencies between calls. Because they follow the same underlying principles, we omit for brevity.

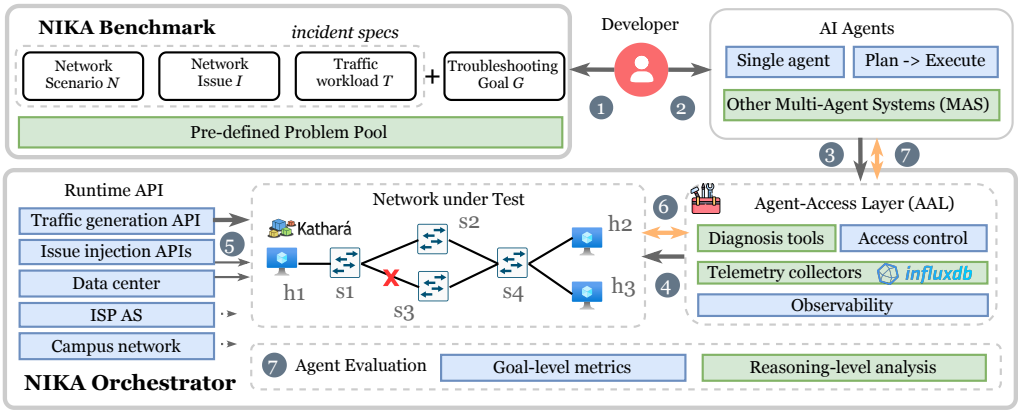


Fig. 2. NIKA's architecture. **Box legend:** blue = provided by NIKA; green = extensible by the developer.

benchmark creation. It consists of two main components: a *benchmark suite* of curated network incidents, and an *orchestration platform* that connects an AI agent with a network environment to reproduce the incident and evaluate the agent's performance.

The developer ❶ selects an incident from the benchmark suite (Sec. 3.2). Each network incident is defined by a specification of how to reproduce the incident, and a troubleshooting goal for the agent ❷ *e.g.,* localize failures. We formalize both in Sec. 3.2.1. Next, the developer interfaces with NIKA's orchestrator (Sec. 3.3) to instantiate the incident in an interactive network-under-test environment and execute the AI agent ❸. Before agent execution, NIKA's orchestrator initializes the network environment based on the incident specification ❹, *e.g.,* deploys the topology, network configurations, and telemetry instrumentation. During agent execution, NIKA exposes ❻ the network environment via an *Agent Access Layer* (AAL) with interfaces that the AI agent can invoke. It also orchestrates traffic generation and failure injection ❺ to faithfully reproduce the incident in the network environment. After agent execution, NIKA evaluates the agent's performance based on goal-specific metrics ❼ (*e.g.,* accuracy for root-cause analysis, time to detection for anomaly detection tasks). It also analyzes the agent's reasoning trajectory, *e.g.,* tool usage patterns, to provide insights into the agent's behavior.

3.2 Benchmark creation: methodology

The user selects a network incident from NIKA's benchmark problem pool to evaluate their AI agent. Each problem in the benchmark is defined by two elements: an *incident specification* that describes how to reproduce the incident in a network environment, and a *troubleshooting specification* that defines the task the agent must perform.

3.2.1 Incident specification. The benchmark is a curated pool of network incidents, each referred as *incident specification* and represented as a three-tuple $(\mathcal{N}, \mathcal{I}, \mathcal{T})$ denoting:

Network scenario $\mathcal{N}$: describes the network topology, the network devices, protocols and configurations involved. It contains initial configurations and the control plane to initialize all nodes in the network. It also defines which telemetry instrumentation to deploy in the network, *e.g.,* INT, sketches, flow collectors, etc, to restrict the use to user-selected telemetry methods. Some legacy clusters may not support advanced switch features such as INT, and users may wish to test how AI agents perform in these clusters. We present the supported network scenarios in appendix A.2.

Network issue $\mathcal{I}$: defines the network problem. $\mathcal{I}$ serves as the ground-truth evidence against which agent outputs are evaluated. Each issue can be further formalized into $\mathcal{I} = (Dev, Comp, RC)$,

where *Dev* is the device with an affected component *Comp* due to a root cause *RC*. For example, a link failure issue can be represented as $\mathcal{I}$ = (`switch1`, `eth0`, `link_down`), indicating that the `eth0` interface on `switch1` is down due to a link failure. Similarly, a routing misconfiguration issue can be represented as $\mathcal{I}$ = (`router2`, `OSPF`, `wrong_area`), indicating that `router2`'s OSPF protocol is misconfigured with an incorrect area assignment. Table 1 categorizes a selected list of currently supported network issues.

Traffic workload $\mathcal{T}$: describes how traffic is generated while reproducing an incident scenario. It includes both regular traffic, which refers to the traffic matrix under normal network operations, and incident-triggering traffic, designed to exacerbate the issue in $\mathcal{I}$. For example, a misconfiguration issue may affect regular traffic, yet manifest with more severe symptoms in the presence of specific incident-triggering traffic patterns that deviate from the average traffic matrix, *e.g.,* overload certain network components. Different combinations of these traffic types allow domain-expert users to modulate troubleshooting complexity by controlling symptom visibility and severity.

Incident variations. Thanks to the incident specification abstraction ($\mathcal{N}, \mathcal{I}, \mathcal{T}$), `NIKA` supports systematic variations of the same underlying issue $\mathcal{I}$ by combining different network scenarios $\mathcal{N}$ and traffic workloads $\mathcal{T}$. Additionally, because $\mathcal{I}$ itself is modular, the user can generate incident variants by swapping affected devices or root causes, *e.g.,* link failure on different switches, link down *vs.* high error rate. This is supported via parametric templates, enabling the generation of practically unlimited incident variants. Alternatively, the user can create composite incident variations by combining multiple issues $\mathcal{I}$ in the same run, *e.g.,* link failure + routing misconfiguration.

3.2.2 Troubleshooting specification. Along with the incident specification, each problem in the `NIKA`'s problem pool is associated with a *troubleshooting goal* $\mathcal{G}$. The AI agent is tasked with $\mathcal{G}$ for the problem at hand. Following the taxonomy used in prior works on cloud diagnosis [57], we adopt a hierarchical structure to define troubleshooting goals, consisting of three levels of increasing complexity: *detection*, *localization*, and *root cause analysis* (RCA). *Detection* is a binary decision of the agent, *i.e.,* normal or anomalous network state. Such a detection task is relevant even in production systems that raise alerts for anomalies to disambiguate false positives and false negatives (which are inevitable). *Localization* identifies the responsible components $\mathcal{I}_{Dev}$ and $\mathcal{I}_{Comp}$ as a multi-class selection task. *RCA* additionally suggests the underlying root causes, $\mathcal{I}_{RC}$. Each level maps to goal-specific evaluators to assess agent performance (Sec. 3.3).

Putting-together example. Figure 3a illustrates how we define the problems in `NIKA`'s problem pool. In this example, the network scenario $\mathcal{N}$ is a data center network (ln.9), and the network issue $\mathcal{I}$ is a resource contention problem due to incast traffic (ln.11). It accepts the affected device and interface as input parameters (ln.8), enabling incident variants. The traffic workload $\mathcal{T}$ consists of uniform traffic (ln.16). The default troubleshooting goal $\mathcal{G}$ is to detect that incast occurred. `NIKA` mandates that the core logic to reproduce the incident is encapsulated in the `run` method (ln.15) to manage the incident lifecycle. This hook will be invoked by `NIKA` orchestrator (Sec. 3.3) once the user connects an AI agent and starts the experiment. Lastly, the problem definition heavily hinges on `NIKA`'s runtime APIs— ln.9,12, and 16, corresponding to *runtime APIs* in Fig. 2 —which we describe next.

3.3 Orchestrator

The orchestrator *consumes* the input incident specification ($\mathcal{N}, \mathcal{I}, \mathcal{T}$), and *materializes* it into execution in a network environment. We walk through the example in Fig. 3b to illustrate how `NIKA` achieves this in practice.

3.3.1 Network environment and Runtime APIs. The user selects and instantiates an incident from the benchmark, with concrete parameters for $\mathcal{N}$ (ln.8 in Fig. 3b). The incident instance is then

registered with the NIKA's orchestrator (ln.9). When the user connects an AI agent (ln.11) and starts the experiment (ln.12), internally the orchestrator invokes the incident's `run()` method (ln.15 in Fig. 3a) to reproduce it. NIKA adopts emulation as a default execution backend because it prioritizes fidelity. However, NIKA's runtime APIs provide high-level abstractions to hide low-level emulation details from the user. For example, NIKA exposes APIs to manage common network topologies (ln.9), deploy and instrument measurement tools in the network nodes (ln.4), and generate traffic matrices (ln.12). It also provides modules to inject network issues. The issue injection (Fig. 2) APIs leverage Linux Traffic Control (TC) for link-level issues, `stress-ng` for software contentions, and custom scripts for device-level failures such as process crashes and misconfigurations.

```python
class IncidentBase:
    prompt_detection = "You are a network engineer.."
    def run(self):
       self.net.deploy().instrument("default")
    # ..
class DataCenterIncast(IncidentBase):
    name = "single_link_datacenter_incast"
    def __init__(self, dev, intf, dcn_size="s"):
       self.network = DataCenterClos(size=dc_size)
       # .. network issue parameters
    def inject_incident(self):
        od = ODFlowGenerator(self.network)
        od.set_src_dst(src="all", dst=dev)
        od.start_traffic(interval=20, speed="X Mbps")
    def run(self):
        UniformTraffic(self.network, rho=.4).start()
        self.inject_incident()
    def eval(self, answer, goal="detect"):
        Evaluator().eval(self, answer, goal)
```

(a) Problem definition in NIKA's benchmark.

```python
from nika.benchmark import DataCenterIncast
from nika.runtime import Orchestrator
from langchain.agents import create_agent
access_policy = {
    "nodes": ["pod0.switch*"];
    "tools" ["*"]
}
prob = DataCenterIncast("s1", "eth0")
NIKA = Orchestrator(env=prob, aal_cfg=access_policy)
tools = NIKA.get_tools()
agent = create_agent("gpt-5", tools=tools)
NIKA.benchmark(agent)
```

(b) Instance creation and agent onboarding.

Fig. 3. NIKA APIs to define and instantiate an incident.

3.3.2 Agent Access Layer (AAL). The Agent Access Layer serves as an access gateway to the network environment for the AI agents. It abstracts the following functionalities.

1) Telemetry collectors & diagnosis tools; AI agents notoriously benefit from structured and well-defined APIs rather than raw CLIs usage and telemetry processing Methods and interfaces available to human engineers (such as dashboards and telemetry backends) are not necessarily ready-to-use by LLM agents. AAL implements well-defined interfaces with structured input-output formats that agents can invoke to observe the network state and perform diagnosis actions. More specifically, NIKA is already equipped with more than 30 MCP tools for active measurements (*e.g.*, ping probing, raw packet dump), passive telemetry (*e.g.*, counter reads, flow queries, INT records and sketches), and telemetry retrieval (*e.g.*, InfluxDB [2]). The detailed list is in Table 5.

2) Agent access policies; NIKA restricts the action scope of AI agents via user-defined polices. The policies reflect network boundaries and a given ownership model. For instance, in a data center scenario, the user may want tenant-side agents to be limited to actions within the tenant's boundary, whereas operator-scoped agents act on the entire fabric. NIKA's policies can be specified in a declarative way (ln.4 in Fig. 3b), and enforce access control at runtime via the AAL (ln.9 in Fig. 3b).

3) Observability. The AAL intercepts every tool call at the boundary between the agent and the network, and provides observability for both tool usage and network state when the agent queries it. For tool usage, the AAL logs each tool invocation with its input parameters, timestamp, and response. For network state, the AAL snapshots the ground-truth telemetry state at the time of each tool invocation, allowing users to inspect agent reasoning in the context of the actual network conditions. Snapshots are persisted beyond the experiment lifetime to support post-mortem querying and analysis. When multiple agents (*e.g.*, tenant-scoped and operator-scoped) interact

Table 1. Selected network issues ($\mathcal{I}$) in the NIKA benchmark. Full incident list is in Table 3.

Category	# Issues	Representative issue	Key Signals
Link failures	6	Link flap	flap event logs; packet drops
End-host failures	10	Conflicting VPN memberships	Overlapping subnets; VPN servers unreachable
Network node errors	8	Number of MPLS labels hit limit	Error logs; packet drops
Misconfigurations	14	BGP ASN mismatch	BGP session fails; ASN mismatch detected
Resource contention	6	Microbursts on interface	Reduced throughput; queue buildup
Network under attack	10	Service DoS	Surge in HTTP connections; CPU/RAM usage spikes

with the same network, the AAL provides a unified view of all tool invocations. NIKA adopts OTel [1] standardized formats to integrate AAL observability with agent reasoning traces, collected by the agent's frameworks.

3.3.3 Evaluator. Finally, NIKA evaluates the agent's performance based on the troubleshooting goal $\mathcal{G}$. NIKA implements goal-specific evaluators for detection, localization, and RCA tasks, and for each of them compares agent outputs against the ground truth $\mathcal{I}$ defined in the incident specification. For localization and RCA, both agent's output and ground truth are represented as boolean masks with entries indicating the presence or absence of faulty devices or root causes. From these lists, the evaluator computes the confusion matrix and then derives the accuracy. Beyond accuracy, NIKA also supports several common efficiency metrics, such as time to detection, token usage, and number of tool invocations. Finally, NIKA tracks flow latency and packet loss during the diagnosis process to quantify side effects on network performance, and reports violations to user-defined SLOs.

4 EXPERIMENTAL RESULTS

4.1 Setup

NIKA implementation. We build NIKA on top of Kathará [12], an open-source container-based network emulator. Traffic workloads are generated via `iperf3` and `ApacheBench`, and the MCP-based AAL is implemented using FastMCP [22] servers. Finally, we collect all agent reasoning trajectories in LangSmith [35] and a local datastore, including tool invocations, LLM prompts, and responses. In total, we release 900 execution traces alongside NIKA.

Agent implementation. We adopt a two-step agentic workflow: the first step executes the troubleshooting tasks and produces an analysis report. The second step then extracts the outputs required to NIKA's evaluator and submits them by invoking the `eval` callback in Fig. 3a. Our prompts are shared in appendix B.2. All agents follow the ReAct paradigm [77] and are implemented in LangGraph. We experiment with two private backend LLMs, GPT-5-mini and GPT-5, via the official OpenAI API, and with a third open-source LLM, GPT-OSS:20B [48], hosted on an NVIDIA GeForce RTX 4090 GPU with 24 GB of VRAM.

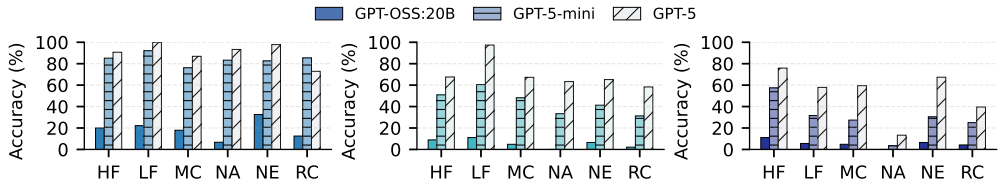
4.2 Performance results

From the full benchmark detailed in appendix A.1, we construct an evaluation subset of 150 incidents. Specifically, we instantiate each issue in one of its typical scenarios. We evaluate three backend models, running each model twice with randomly selected faulty devices ($\mathcal{I}.Dev$) and components ($\mathcal{I}.Comp$) per run. Under this setup, executing the full evaluation requires approximately 7 hours for GPT-OSS:20B, 10 hours for GPT-5-mini and 15 hours for GPT-5 on an Ubuntu 22.04 VM with 16 CPU cores and 32 GB of RAM.

Even with SOTA LLMs, troubleshooting is challenging without specialized agent designs. We report the results in Table 2. As expected, GPT-5 consistently outperforms alternatives across all troubleshooting objectives, improving detection w.r.t. GPT-5-mini by roughly 20%, localization by nearly 2×, and RCA accuracy by more than 2.5×. These improvements are even more pronounced

Table 2. Performance on NIKA benchmark for different LLMs (columns are averages).

Model	Time (s)	# Steps	# Tools	# In tokens	# Out tokens	# Rea. Tokens	# Det. Acc.	# Loc. Acc.	# RCA Acc.
GPT-OSS:20B	175.8	12.6	11.7	69267.1	7645.3	-	19.0%	5.5%	5.5%
GPT-5-mini	242.6	9.5	15.0	156050.1	6578.6	4532.7	74.0%	36.0%	22.0%
GPT-5	359.2	7.0	28.4	105171.9	14603.0	12352.4	89.0%	68.7%	55.3%



(a) Detection (b) Localization (c) RCA

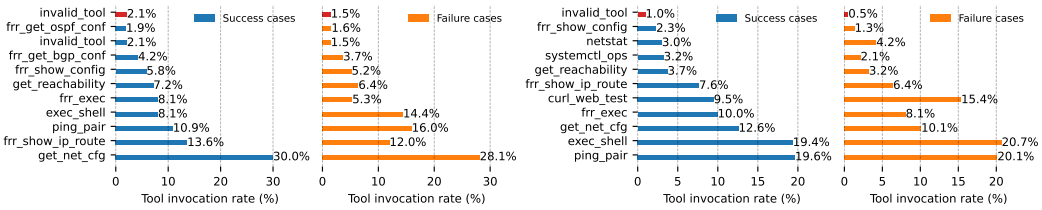
Fig. 4. GPT-5 agent versus network issues type (Table 1). **Legend**: ***LF***: Link Failure, ***NE:*** Network node Error, ***NA:*** Network under Attack, ***EF:*** End-host Failure, ***MC:*** MisConfiguration, ***RC:*** Resource Contention.

when compared against GPT-OSS:20B. Besides better accuracy, GPT-5 exhibits qualitatively stronger reasoning behavior: it issues more tool invocations while requiring fewer reasoning steps, consumes fewer input tokens (105k *vs.* 156k), and produces substantially more output tokens (14.6k *vs.* 6.5k) and reasoning budget (12.3k *vs.* 4.5k tokens). These improvements come at a higher average per-incident runtime, which increases by ≈ 50% when moving from GPT-5-mini to GPT-5.

However, even for GPT-5, the overall performance remains far from perfect. To better understand the limitations of current LLMs for network troubleshooting, we analyze the performance of the three LLMs across different issue categories in Fig. 4. GPT-5 maintains high detection accuracy across the majority of issue categories, with resource contention as the main exception. Localization and RCA exhibit an even larger variation: link failure (LF) achieves the highest localization accuracy (97%), while resource contention remains challenging (58%). We manually investigated the agent reasoning traces and network telemetry, recorded by NIKA's observability module, and observed that for incidents with less direct symptoms, the agent tends to stop at shallow, connectivity-centric explanations. For example, in resource contention-related problems (*e.g.,* `link_packet_corruption`, `NF_load_balancer_overload`, `sender_resource_contention`), despite persistent indicators such as CRC errors, queue drops, and asymmetric losses, the agent settles on vague diagnoses such as "link unstable" or "transient congestion", without connecting these symptoms to the deeper causes.

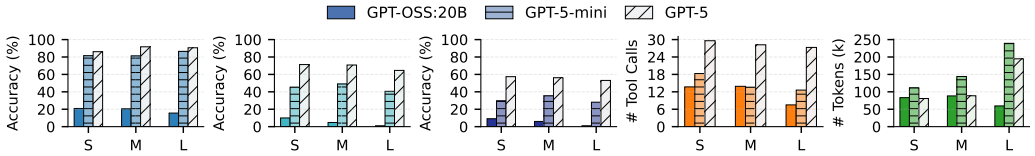
Tool usage analysis. Additionally, we analyze tool invocation patterns to understand inefficiencies. Fig. 5 compares GPT-5-mini and GPT-5 across success cases (all goals $\mathcal{G}$ achieved) and failure cases (at least one goal failed). In the successful cases, GPT-5-mini (Fig. 5a) relies mostly on high-level checks, *e.g.,* `get_host_net_config`, remarking that the model can manage troubleshooting at the network layer. GPT-5 (Fig. 5b), in contrast, makes broader use of application-layer diagnostics, such as `exec_shell` and `curl_web_test`, reflecting its ability to seek out issues across multiple layers. Given that many issues in our benchmark affect application performance without fully breaking connectivity, GPT-5-mini's limited use of higher-layer tools directly contributes to its weaker diagnostic performance. Lastly, this application-layer diagnostic appears more frequently in GPT-5-mini's failed runs, suggesting that while the model recognizes its relevance, it fails to incorporate it effectively into its reasoning process.

To quantify erroneous tool invocations, we capture and count the exceptions raised during tool calls, including parameter validation errors at the MCP client and execution failures at the MCP server. As shown by the top red bars in Fig. 5, both models exhibit remarkably low error rates, averaging 1.6% for GPT-5-mini and 0.7% for GPT-5 across all success and failure cases. These errors are typically minor parameter mismatches, which contrasts with the significant tool



(a) GPT-5-mini (b) GPT-5

Fig. 5. Tool invocation distribution comparison between successful and failed troubleshooting.



(a) Detection (b) Localization (c) RCA (d) # Tool Calls (e) # Tokens

Fig. 6. Agent scalability versus network topology size.

hallucination issues reported in prior work [18, 57], underscoring the importance of structured and well-documented MCP-based interfaces in the AAL.

Sensitivity to topology size. Finally, we evaluate the impact of network size by using the NIKA runtime API to instantiate small (S), medium (M), and large (L) topologies for four scenarios: data center, campus, ISP backbone, and SDN-enabled cloud POP fabric. On average, these topologies contain 11, 27, and 101 nodes, respectively. Fig. 6 summarizes our findings. Detection accuracy remains relatively stable across scales, indicating that anomalous symptoms remain visible as networks grow. In contrast, localization and RCA degrade with size, reflecting the ambiguity introduced by larger and more interconnected environments. Although tool invocations decrease slightly with scale, total token consumption nearly doubles for GPT-5-mini and GPT-5, suggesting that larger topologies force agents to process substantially more context, making precise fault isolation increasingly difficult.

5 DISCUSSION & LIMITATIONS

We surface potential use cases and highlight areas for improvements.

NIKA as an educational platform. The rise of LLMs is transforming networking education, with faculty grappling with questions about responsible AI use and how to foster deep understanding in an era of AI-assisted learning. NIKA can support networking education by exposing students to systematic troubleshooting through a curated incident repository and AI-generated reasoning traces. Instructors can design assignments where students analyze agent behavior, compare their own diagnostics against AI outputs, and identify failure modes. This enables teaching both networking fundamentals and the limitations of current LLM-based systems. We publicly release 900+ reasoning traces to facilitate this use case.

Network backend. NIKA inherits the limits of network emulation: it cannot faithfully reproduce issues that manifest only in high-speed networks [81], or that require specialized hardware. However, we believe that a significant number of issues can still be meaningfully studied in emulated environments by scaling down link speeds, traffic loads, and the time resolution of network measurements, while preserving the core causal relationships and the fidelity of troubleshooting

tools. Nevertheless, NIKA's architecture itself remains environment-agnostic and only assumes that a network backend can reproduce $(\mathcal{N}, \mathcal{I}, \mathcal{T})$ and expose tools through a common interface, so alternative backends (*e.g.,* simulators with realistic adapters) are feasible but left to future work.

Mitigation and safe agent actions. NIKA currently focuses on diagnosis tasks (detection, localization, RCA) but does not yet support evaluating mitigation actions. Enabling this capability presents non-trivial challenges: evaluating whether an agent-proposed remediation is correct requires both checking syntactic correctness and, crucially, assessing whether its execution would be safe and non-disruptive to the network. To address this, we plan to integrate Batfish [17] as an additional tool in the AAL, enabling agents to validate candidate remediation plans against the network model before applying them. This will allow future versions of NIKA to provide automated verification of mitigation strategies without risking the emulated network.

6 RELATED WORK

Network monitoring. A long line of systems has advanced telemetry and diagnosis for large-scale networks [8, 11, 20, 24, 27, 28, 30, 32, 46, 52, 59, 61, 65]. These works provide rich measurement substrates and partial automation for operators, whereas NIKA targets the complementary problem of autonomous decision-making troubleshooting agents that must choose tools, interpret heterogeneous telemetry, and synthesize diagnoses.

LLMs for network operations. Microsoft first outlined the vision of AI-driven network incident management for the Azure cloud [29]. BIAN [63] assists operators at Alibaba Cloud in localizing faulty devices across large-scale WANs using telemetry and historical incident data. Confucius [68] generalizes this idea to multi-agent LLM-based intent-driven management for Meta's networks, while NetAssistant [64] targets on-call support for ByteDance. These systems validate the feasibility of LLM-powered network copilots in production, but do not offer an open, reusable benchmark and execution environment for troubleshooting agents.

Benchmarks for LLMs in networking. NetConfEval [62] and NetLLMBench [9] are public benchmark for network configuration synthesis from natural language intents. They focus on static configuration tasks, whereas NIKA targets dynamic troubleshooting in live network environments. NetPress [85], similar to NIKA, proposes dynamically generated LLM benchmarks for network applications, however it focuses on network management at large, while NIKA specifically targets troubleshooting tasks. Moreover, it lacks an end-to-end, reusable toolchain that enables accessible experimentation for a broad research and practitioner community. By contrast, NIKA provides APIs for several network scenarios, fault injection, tool integration, traffic generation and observability.

LLM research for DevOps. Beyond networking, LLMs have been used to analyze telemetry data [4, 5, 42, 56, 73, 79], perform root-cause analysis in cloud systems [15, 19, 37, 55, 57, 78], and assist with software engineering [13, 14, 31, 72, 74]. Network troubleshooting shares similarities with these domains, and we expect NIKA to facilitate cross-pollination of ideas and techniques.

7 CONCLUSION

We designed NIKA, a unified benchmarking framework for AI-driven network troubleshooting. NIKA offers a curated suite of incident scenarios covering a broad spectrum of real-world faults and network topologies, along with an orchestration platform that connects agents to network environments and exposes their behavior for evaluation. It requires minimal user involvement beyond selecting a network incident from the benchmark suite and providing the AI agent implementation, yet its modular architecture allows advanced users to customize and extend each component. We see NIKA as a first step in accelerating innovation in AI-driven network diagnosis, and we hope for community contributions to expand it to cover more scenarios and network types.

References

[1] 2025. OpenTelemetry. https://opentelemetry.io/.

[2] Khaleel Ahmad and Masroor Ansari. 2017. Hands-on influxdb. In *NoSQL*. Chapman and Hall/CRC, 341–354.

[3] Weissberger Alan. 2025. Ericsson integrates agentic AI into its NetCloud platform for self-healing and autonomous 5G private networks. https://techblog.comsoc.org/2025/09/12/ericsson-integrates-agentic-ai-into-its-netcloud-platform-for-self-healing-and-autonomous-5g-private-networks/. Accessed: 2025-12-10.

[4] Sarah Alnegheimish, Linh Nguyen, Laure Berti-Equille, and Kalyan Veeramachaneni. 2024. Large language models can be zero-shot anomaly detectors for time series? arXiv:2405.14755 [cs.LG] https://arxiv.org/abs/2405.14755

[5] Vaastav Anand, Pedro Las-Casas, Rodrigo Fonseca, and Antoine Kaufmann. 2025. Towards Using Llms for Distributed Trace Comparison (Abstract) . In *2025 IEEE/ACM International Workshop on Cloud Intelligence & AIOps (AIOps)*. IEEE Computer Society.

[6] Antonino Angi, Alessio Sacco, and Guido Marchetto. 2025. LLNet: An Intent-Driven Approach to Instructing Softwarized Network Devices Using a Small Language Model. *IEEE Transactions on Network and Service Management* 22, 4 (2025), 3403–3418.

[7] Anthropic. 2025. Model Context Protocol. https://modelcontextprotocol.io. Accessed: 2025-12-11.

[8] Behnaz Arzani, Selim Ciraci, Luiz Chamon, Yibo Zhu, Hongqiang Harry Liu, Jitu Padhye, Boon Thau Loo, and Geoff Outhred. 2018. 007: Democratically finding the cause of packet drops. In *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI '18)*. USENIX Association, 419–435.

[9] Kaan Aykurt, Andreas Blenk, and Wolfgang Kellerer. 2024. NetLLMBench: A Benchmark Framework for Large Language Models in Network Configuration Tasks. In *2024 IEEE Conference on Network Function Virtualization and Software Defined Networks (NFV-SDN'24)*. 1–6.

[10] Kinjal Basu, Ibrahim Abdelaziz, Kiran Kate, Mayank Agarwal, Maxwell Crouse, Yara Rizk, Kelsey Bradford, Asim Munawar, Sadhana Kumaravel, Saurabh Goyal, Xin Wang, Luis A. Lastras, and Pavan Kapanipathi. 2025. NESTFUL: A Benchmark for Evaluating LLMs on Nested Sequences of API Calls. arXiv:2409.03797 [cs.AI]

[11] Ran Ben Basat, Sivaramakrishnan Ramanathan, Yuliang Li, Gianni Antichi, Minian Yu, and Michael Mitzenmacher. 2020. PINT: Probabilistic in-band network telemetry. In *Proceedings of the ACM SIGCOMM 2020 Conference (SIGCOMM '20)*. Association for Computing Machinery, 662–680.

[12] Gaetano Bonofiglio, Veronica Iovinella, Gabriele Lospoto, and Giuseppe Di Battista. 2018. Kathará: A container-based framework for implementing network function virtualization and software defined networks. In *Proceedings of 2018 IEEE/IFIP Network Operations and Management Symposium (NOMS'18)*. IEEE, 1–9.

[13] Bei Chen, Shuai Zhang, Jianfeng Zhang, Cyril Wang, Renjie Zhou, Yang song Lyu, Bei Su, Lifan Wang, Yan Zhang, Zhaojun Liu, et al. 2022. CodeT: Code Generation with Generated Tests. *arXiv preprint arXiv:2207.10397* (2022).

[14] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. 2024. Teaching Large Language Models to Self-Debug. In *ICLR*. https://openreview.net/forum?id=KuPixIqPiq

[15] Yinfang Chen, Huaibing Xie, Minghua Ma, Yu Kang, Xin Gao, Liu Shi, Yunjie Cao, Xuedong Gao, Hao Fan, Ming Wen, et al. 2024. Automatic root cause analysis via large language models for cloud incidents. In *Proceedings of the Nineteenth European Conference on Computer Systems (EuroSys'24)*. Association for Computing Machinery, 674–688.

[16] Google Cloud. 2025. Shaping the future. The transofrmative potential of agentic AI and the strategic imperative for Google Cloud partners. https://cloud.google.com/blog/topics/partners/sharing-new-report-on-the-potential-of-agentic-ai. Accessed: 2025-12-10.

[17] Batfish Project Contributors. 2025. Batfish. https://github.com/batfish/batfish.

[18] Alessandro Cornacchia, Iliyas Alabdulaal, Ibraheem Saghier, Albaraa Mirdad, Omar Fayoumi, and Marco Canini. 2025. Between Promise and Pain: The Reality of Automating Failure Analysis in Microservices with LLMs. In *Proceedings of the 16th ACM SIGOPS Asia-Pacific Workshop on Systems (APSys '25)*. Association for Computing Machinery.

[19] Alessandro Cornacchia, Iliyas Alabdulaal, Ibraheem Saghier, Albaraa Mirdad, Omar Fayoumi, and Marco Canini. 2025. Between Promise and Pain: The Reality of Automating Failure Analysis in Microservices with LLMs. In *Proceedings of the 16th ACM SIGOPS Asia-Pacific Workshop on Systems (APSys '25)*. Association for Computing Machinery, 155–167.

[20] Amogh Dhamdhere, Renata Teixeira, Constantine Dovrolis, and Christophe Diot. 2007. NetDiagnoser: Troubleshooting network unreachabilities using end-to-end probes and routing data. In *Proceedings of the 2007 ACM CoNEXT conference (CoNEXT '07)*. Association for Computing Machinery, 1–12.

[21] Denis Donadel, Francesco Marchiori, Luca Pajola, and Mauro Conti. 2024. Can LLMs Understand Computer Networks? Towards a Virtual System Administrator. In *2024 IEEE 49th Conference on Local Computer Networks (LCN '24)*. IEEE, 1–10.

[22] FastMCP. 2025. FastMCP: The fast, Pythonic way to build MCP servers and clients. https://github.com/jlowin/fastmcp. Accessed: 2025-12-07.

[23] Seyed K Fayaz, Tushar Sharma, Ari Fogel, Ratul Mahajan, Todd Millstein, Vyas Sekar, and George Varghese. 2016. Efficient network reachability analysis using a succinct control plane representation. In *12th USENIX Symposium on*

Operating Systems Design and Implementation (OSDI '16). USENIX Association, 217–232.

[24] Yilong Geng, Shiyu Liu, Zi Yin, Ashish Naik, Balaji Prabhakar, Mendel Rosenblum, and Amin Vahdat. 2019. SIMON: A Simple and Scalable Method for Sensing, Inference and Measurement in Data Center Networks. In *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI '19)*. USENIX Association, 549–564.

[25] Drishti Goel, Fiza Husain, Aditya Singh, Supriyo Ghosh, Anjaly Parayil, Chetan Bansal, Xuchao Zhang, and Saravan Rajmohan. 2024. X-Lifecycle Learning for Cloud Incident Management using LLMs. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering (FSE '24)*. Association for Computing Machinery, 417–428.

[26] Fengchen Gong, Divya Raghunathan, Aarti Gupta, and Maria Apostolaki. 2024. Zoom2Net: Constrained Network Telemetry Imputation. In *Proceedings of the ACM SIGCOMM 2024 Conference (ACM SIGCOMM '24)*. Association for Computing Machinery, 764–777.

[27] Chuanxiong Guo, Lihua Yuan, Dong Xiang, Yingnong Dang, Ray Huang, Dave Maltz, Zhaoyi Liu, Vin Wang, Bin Pang, Hua Chen, et al. 2015. Pingmesh: A large-scale system for data center network latency measurement and analysis. In *Proceedings of the ACM SIGCOMM 2015 Conference (SIGCOMM '15)*. Association for Computing Machinery, 139–152.

[28] Arpit Gupta, Rob Harrison, Marco Canini, Nick Feamster, Jennifer Rexford, and Walter Willinger. 2018. Sonata: Query-driven streaming network telemetry. In *Proceedings of the ACM SIGCOMM 2018 Conference (SIGCOMM '18)*. Association for Computing Machinery, 357–371.

[29] Pouya Hamadanian, Behnaz Arzani, Sadjad Fouladi, Siva Kesava Reddy Kakarla, Rodrigo Fonseca, Denizcan Billor, Ahmad Cheema, Edet Nkposong, and Ranveer Chandra. 2023. A Holistic View of AI-driven Network Incident Management. In *ACM HotNets*. Association for Computing Machinery.

[30] Nikhil Handigol, Brandon Heller, Vimalkumar Jeyakumar, David Mazières, and Nick McKeown. 2014. I know what your packet did last hop: Using packet histories to troubleshoot networks. In *11th USENIX Symposium on Networked Systems Design and Implementation (NSDI '14)*. USENIX Association, 71–85.

[31] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. 2024. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. In *ICLR*. https://openreview.net/forum?id=VtmBAGCN7o

[32] Qun Huang, Haifeng Sun, Patrick PC Lee, Wei Bai, Feng Zhu, and Yungang Bao. 2020. Omnimon: Re-architecting network telemetry with resource efficiency and full accuracy. In *Proceedings of the ACM SIGCOMM 2020 Conference (SIGCOMM '20)*. Association for Computing Machinery, 404–421.

[33] Jinxi Kuang, Jinyang Liu, Junjie Huang, Renyi Zhong, Jiazhen Gu, Lan Yu, Rui Tan, Zengyin Yang, and Michael R Lyu. 2024. Knowledge-aware alert aggregation in large-scale cloud systems: a hybrid approach. In *Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP '24)*. Association for Computing Machinery, 369–380.

[34] LangChain. 2025. LangGraph: Workflows and Agents. https://langchain-ai.github.io/langgraph/tutorials/workflows/#set-up. Accessed: 2025-12-10.

[35] LangChain. 2025. LangSmith Observability. https://www.langchain.com/langsmith. Accessed: 2025-12-10.

[36] Jonatan Langlet, Ran Ben Basat, Gabriele Oliaro, Michael Mitzenmacher, Minlan Yu, and Gianni Antichi. 2023. Direct Telemetry Access. In *Proceedings of the ACM SIGCOMM 2023 Conference (SIGCOMM '23)*. Association for Computing Machinery, 832–849.

[37] Pedro Las-Casas, Alok Gautum Kumbhare, Rodrigo Fonseca, and Sharad Agarwal. 2024. LLexus: an AI agent system for incident management. *SIGOPS Oper. Syst. Rev.* 58, 1 (Aug. 2024).

[38] Kuang-Huei Lee, Xinyun Chen, Hiroki Furuta, John Canny, and Ian Fischer. 2024. A human-inspired reading agent with gist memory of very long contexts. In *Proceedings of the 41st International Conference on Machine Learning (ICML '24)*. JMLR.org, 26396–26415.

[39] Bingdong Li, Jeff Springer, George Bebis, and Mehmet Hadi Gunes. 2013. A survey of network flow applications. *Journal of Network and Computer Applications* 36, 2 (2013), 567–581.

[40] Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. 2023. CAMEL: communicative agents for "mind" exploration of large language model society. In *Proceedings of the 37th International Conference on Neural Information Processing Systems (NIPS '23)*. Curran Associates Inc., 51991–52008.

[41] Oscar G. Lira, Oscar M. Caicedo, and Nelson L. S. da Fonseca. 2025. Large Language Models for Zero Touch Network Configuration Management. *IEEE Communications Magazine* 63, 7 (2025), 146–153.

[42] Jun Liu, Chaoyun Zhang, Jiaxu Qian, Minghua Ma, Si Qin, Chetan Bansal, Qingwei Lin, Saravan Rajmohan, and Dongmei Zhang. 2024. Large Language Models can Deliver Accurate and Interpretable Time Series Anomaly Detection. arXiv:2405.15370 [cs.CL] https://arxiv.org/abs/2405.15370

[43] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck,

Amir Yazdanbakhsh, and Peter Clark. 2023. SELF-REFINE: iterative refinement with self-feedback. In *Proceedings of the 37th International Conference on Neural Information Processing Systems (NIPS '23)*. Curran Associates Inc., 46534–46594.

[44] Abdelkader Mekrache, Adlen Ksentini, and Christos Verikoukis. 2024. Intent-based management of next-generation networks: An LLM-centric approach. *IEEE Network* 38, 5 (2024), 29–36.

[45] Justin Meza, Tianyin Xu, Kaushik Veeraraghavan, and Onur Mutlu. 2018. A Large Scale Study of Data Center Network Reliability. In *Proceedings of the Internet Measurement Conference 2018 (IMC '18)*. Association for Computing Machinery, New York, NY, USA.

[46] Masoud Moshref, Minlan Yu, Ramesh Govindan, and Amin Vahdat. 2016. Trumpet: Timely and precise triggers in data centers. In *Proceedings of the 2016 ACM SIGCOMM Conference (SIGCOMM '16)*. Association for Computing Machinery, 129–143.

[47] Hun Namkung, Zaoxing Liu, Daehyeok Kim, Vyas Sekar, and Peter Steenkiste. 2023. Sketchovsky: Enabling Ensembles of Sketches on Programmable Switches. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI '23)*. USENIX Association, 1273–1292.

[48] OpenAI. 2025. gpt-oss series, OpenAI's open-weight models. https://github.com/openai/gpt-oss. Accessed: 2025-12-10.

[49] P4.org. 2020. P4 In-band Network Telemetry (INT) Specification. https://p4.org/p4-spec/docs/INT_v2_1.pdf. Accessed: 2025-05-30.

[50] Melissa Z Pan, Mert Cemri, Lakshya A Agrawal, Shuyi Yang, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Kannan Ramchandran, Dan Klein, Joseph E. Gonzalez, Matei Zaharia, and Ion Stoica. 2025. Why Do Multiagent Systems Fail?. In *ICLR 2025 Workshop on Building Trust in Language Models and Applications (ICLR '25)*.

[51] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. 2024. Gorilla: Large Language Model Connected with Massive APIs. In *Advances in Neural Information Processing Systems (NIPS '24)*, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (Eds.). Curran Associates, Inc., 126544–126565.

[52] Yanghua Peng, Ji Yang, Chuan Wu, Chuanxiong Guo, Chengchen Hu, and Zongpeng Li. 2017. deTector: a Topology-aware Monitoring System for Data Center Networks. In *2017 USENIX Annual Technical Conference (ATC '17)*. USENIX Association, 55–68.

[53] M. Peuster, H. Karl, and S. van Rossem. 2016. MeDICINE: Rapid prototyping of production-ready network services in multi-PoP environments. In *Proceedings of 2016 IEEE Conference on Network Function Virtualization and Software Defined Networks (NFV-SDN'16)*. IEEE, 148–153.

[54] Peter Phaal, Sonia Panchen, and Neil McKee. 2001. *InMon corporation's sFlow: A method for monitoring traffic in switched and routed networks*. Technical Report.

[55] Devjeet Roy, Xuchao Zhang, Rashi Bhave, Chetan Bansal, Pedro Las-Casas, Rodrigo Fonseca, and Saravan Rajmohan. 2024. Exploring LLM-Based Agents for Root Cause Analysis. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering (FSE '24)*. Association for Computing Machinery, 208–219.

[56] Vishwanath Seshagiri, Siddharth Balyan, Vaastav Anand, Kaustubh Dhole, Ishan Sharma, Avani Wildani, José Cambronero, and Andreas Züfle. 2024. Chatting with Logs: An exploratory study on Finetuning LLMs for LogQL. arXiv:2412.03612 [cs.DB] https://arxiv.org/abs/2412.03612

[57] Manish Shetty, Yinfang Chen, Gagan Somashekar, Minghua Ma, Yogesh Simmhan, Xuchao Zhang, Jonathan Mace, Dax Vandevoorde, Pedro Las-Casas, Shachee Mishra Gupta, Suman Nath, Chetan Bansal, and Saravan Rajmohan. 2024. Building AI Agents for Autonomous Clouds: Challenges and Design Principles. In *Proceedings of the 2024 ACM Symposium on Cloud Computing (SoCC '24)*. Association for Computing Machinery, 99–110.

[58] SRL-Labs. 2020. Containerlab: Container-Based Networking Labs. https://containerlab.dev/. Accessed: 2025-05-30.

[59] Cheng Tan, Ze Jin, Chuanxiong Guo, Tianrong Zhang, Haitao Wu, Karl Deng, Dongming Bi, and Dong Xiang. 2019. NetBouncer: Active Device and Link Failure Localization in Data Center Networks. In *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI '19)*. USENIX Association, 599–614.

[60] Mininet Team. 2011. Mininet: An Instant Virtual Network on your Laptop (or other PC). https://mininet.org/. Accessed: 2025-05-30.

[61] TONG Van, Hai Anh Tran, Sami Souihi, and Abdelhamid Mellouk. 2018. Network troubleshooting: Survey, taxonomy and challenges. In *2018 International Conference on Smart Communications in Network Technologies (SaCoNeT '18)*. IEEE, 165–170.

[62] Changjie Wang, Mariano Scazzariello, Alireza Farshin, Simone Ferlin, Dejan Kostić, and Marco Chiesa. 2024. NetConfEval: Can LLMs Facilitate Network Configuration? *Proceedings of the ACM on Networking* 2, CoNEXT2 (2024), 1–25.

[63] Chenxu Wang, Xumiao Zhang, Runwei Lu, Xianshang Lin, Xuan Zeng, Xinlei Zhang, Zhe An, Gongwei Wu, Jiaqi Gao, Chen Tian, Guihai Chen, Guyue Liu, Yuhong Liao, Tao Lin, Dennis Cai, and Ennan Zhai. 2025. Towards LLM-Based Failure Localization in Production-Scale Networks. In *Proceedings of the ACM SIGCOMM 2025 Conference (SIGCOMM '25)*. Association for Computing Machinery, 496–511.

[64] Haopei Wang, Anubhavnidhi Abhashkumar, Changyu Lin, Tianrong Zhang, Xiaoming Gu, Ning Ma, Chang Wu, Songlin Liu, Wei Zhou, Yongbin Dong, et al. 2024. NetAssistant: Dialogue based network diagnosis in data center networks. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI '24)*. USENIX Association, 2011–2024.

[65] Weitao Wang, Xinyu Crystal Wu, Praveen Tammana, Ang Chen, and T. S. Eugene Ng. 2022. Closed-loop Network Performance Monitoring and Diagnosis with SpiderMon. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI '22)*. USENIX Association, 267–285.

[66] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. Self-consistency improves chain of thought reasoning in language models. In *ICLR*. OpenReview.net.

[67] Yidi Wang, Yue Pang, Yanli Liu, Yao Zhang, Lifang Zhang, Min Zhang, and Danshi Wang. 2025. Graph Structure-Enhanced Large Language Model for Optical Network Fault Diagnosis: An Explainable Alarm Root Cause Localization Approach. *IEEE Internet of Things Journal* (2025), 31493–31510.

[68] Zhaodong Wang, Samuel Lin, Guanqing Yan, Soudeh Ghorbani, Minlan Yu, Jiawei Zhou, Nathan Hu, Lopa Baruah, Sam Peters, Srikanth Kamath, et al. 2025. Intent-Driven Network Management with Multi-Agent LLMs: The Confucius Framework. In *Proceedings of the ACM SIGCOMM 2025 Conference (SIGCOMM '25)*. Association for Computing Machinery, 347–362.

[69] Zefan Wang, Zichuan Liu, Yingying Zhang, Aoxiao Zhong, Jihong Wang, Fengbin Yin, Lunting Fan, Lingfei Wu, and Qingsong Wen. 2024. RCAgent: Cloud Root Cause Analysis by Autonomous Agents with Tool-Augmented Large Language Models. In *ACM Conference on Information and Knowledge Management (CIKM '24)*. Association for Computing Machinery.

[70] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. In *Proceedings of the 36th International Conference on Neural Information Processing Systems (NIPS '22)*. Curran Associates Inc., 24824–24837.

[71] Duo Wu, Xianda Wang, Yaqi Qiao, Zhi Wang, Junchen Jiang, Shuguang Cui, and Fangxin Wang. 2024. NetLLM: Adapting Large Language Models for Networking. In *Proceedings of the ACM SIGCOMM 2024 Conference (SIGCOMM '24)*. Association for Computing Machinery, 661–678.

[72] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2025. Demystifying LLM-Based Software Engineering Agents. *Proc. ACM Softw. Eng.* 2, FSE (June 2025). https://doi.org/10.1145/3715754

[73] Zhe Xie, Zeyan Li, Xiao He, Longlong Xu, Xidao Wen, Tieying Zhang, Jianjun Chen, Rui Shi, and Dan Pei. 2025. ChatTS: Aligning Time Series with LLMs via Synthetic Data for Enhanced Understanding and Reasoning. arXiv:2412.03104 [cs.AI] https://arxiv.org/abs/2412.03104

[74] Junjielong Xu, Qinan Zhang, Zhiqing Zhong, Shilin He, Chaoyun Zhang, Qingwei Lin, Dan Pei, Pinjia He, Dongmei Zhang, and Qi Zhang. 2025. OpenRCA: Can Large Language Models Locate the Root Cause of Software Failures?. In *ICRL*. https://openreview.net/forum?id=M4qNIzQYpd

[75] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2025. A-MEM: Agentic Memory for LLM Agents. arXiv:2502.12110 [cs.CL]

[76] Tong Yang, Jie Jiang, Peng Liu, Qun Huang, Junzhi Gong, Yang Zhou, Rui Miao, Xiaoming Li, and Steve Uhlig. 2018. Elastic sketch: Adaptive and fast network-wide measurements. In *Proceedings of the ACM SIGCOMM 2018 Conference (SIGCOMM '18)*. Association for Computing Machinery, 561–575.

[77] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *Proceedings of the 11th International Conference on Learning Representations (ICLR '23)*.

[78] Zhaoyang Yu, Minghua Ma, Chaoyun Zhang, Si Qin, Yu Kang, Chetan Bansal, Saravan Rajmohan, Yingnong Dang, Changhua Pei, Dan Pei, Qingwei Lin, and Dongmei Zhang. 2024. MonitorAssistant: Simplifying Cloud Service Monitoring via Large Language Models. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering (FSE'24)*. Association for Computing Machinery, 38–49.

[79] Chenxi Zhang, Bicheng Zhang, Dingyu Yang, Xin Peng, Miao Chen, Senyu Xie, Gang Chen, Wei Bi, and Wei Li. 2025. PromAssistant: Leveraging Large Language Models for Text-to-PromQL. arXiv:2503.03114 [cs.SE] https://arxiv.org/abs/2503.03114

[80] Dylan Zhang, Xuchao Zhang, Chetan Bansal, Pedro Las-Casas, Rodrigo Fonseca, and Saravan Rajmohan. 2024. LM-PACE: Confidence estimation by large language models for effective root causing of cloud incidents. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering (FSE '24)*. Association for Computing Machinery, 388–398.

[81] Qiao Zhang, Vincent Liu, Hongyi Zeng, and Arvind Krishnamurthy. 2017. High-resolution measurement of data center microbursts. In *Proceedings of the 2017 Internet Measurement Conference (IMC '17)*. Association for Computing Machinery, New York, NY, USA.

[82] Nengwen Zhao, Junjie Chen, Xiao Peng, Honglin Wang, Xinya Wu, Yuanzong Zhang, Zikai Chen, Xiangzhong Zheng, Xiaohui Nie, Gang Wang, et al. 2020. Understanding and handling alert storm for online service systems. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP '20)*. Association for Computing Machinery, 162–171.

[83] Yikai Zhao, Kaicheng Yang, Zirui Liu, Tong Yang, Li Chen, Shiyi Liu, Naiqian Zheng, Ruixin Wang, Hanbo Wu, Yi Wang, et al. 2021. LightGuardian: A Full-Visibility, Lightweight, In-band Telemetry System Using Sketchlets. In *18th USENIX symposium on networked systems design and implementation (NSDI '21)*. USENIX Association, 991–1010.

[84] Wang Zhihao, Cornacchia Alessandro, Sacco Alessio, Galange Franco, and Canini Marco. 2025. NIKA: Network Incidents Benchmarking Framework for AI Agents. GitHub repository: https://github.com/zhihao1998/LLM4NetLab.

[85] Yajie Zhou, Jiajun Ruan, Eric S Wang, Sadjad Fouladi, Francis Y Yan, Kevin Hsieh, and Zaoxing Liu. 2025. NetPress: Dynamically Generated LLM Benchmarks for Network Applications. *arXiv preprint arXiv:2506.03231* (2025).

[86] Yu Zhou, Chen Sun, Hongqiang Harry Liu, Rui Miao, Shi Bai, Bo Li, Zhilong Zheng, Lingjun Zhu, Zhen Shen, Yongqing Xi, Pengcheng Zhang, Dennis Cai, Ming Zhang, and Mingwei Xu. 2020. Flow Event Telemetry on Programmable Data Plane. In *Proceedings of the ACM SIGCOMM 2020 Conference (SIGCOMM '20)*. Association for Computing Machinery, 76–89.

[87] Mingchen Zhuge, Wenyi Wang, Louis Kirsch, Francesco Faccio, Dmitrii Khizbullin, and Jürgen Schmidhuber. 2024. GPTSwarm: Language Agents as Optimizable Graphs. In *Proceedings of the 41st International Conference on Machine Learning (ICML '24)*. PMLR, 62743–62767.

Table 3. Network issues ($\mathcal{I}$) and related incidents in the NIKA benchmark.

Category	Root Cause	Key Signals	# Incident
Link failures	Link flap	flap event logs; packet drops	26
	Link detached	Physical link not detected; PHY down	26
	Link down	Interface state down	26
	Faulty cable	CRC errors; corrupted frames	26
	MAC address conflict	Same MAC seen on multiple ports; MAC flapping logs	26
	Link fragmentation disabled	Large packets dropped; MTU mismatch	26
End-host failures	Conflicting VPN memberships	Overlapping subnets; VPN servers unreachable	3
	Host crash	Host unresponsive; no heartbeat; ping fails	35
	Host IP conflict	Duplicate IP alerts; ARP conflict detected	26
	Host IP misconfig	Incorrect or missing IP address; host unresponsive	68
	Incorrect netmask	Partial reachability; inconsistent routing behavior	16
	DNS empty answer	Incorrect or missing DNS records; NXDOMAIN	6
Network node errors	Number of MPLS labels hit limit	Error logs; packet drops	1
	Switch/router crash (e.g., overheating)	Switch down and unreachable from MGMT	20
	P4 program reads `invalid` header field	Packet drops; error logs (plaform-dependent)	8
	SDN controller crash	Switches isolated; new flows dropped	6
	Southbound port unreachable	OpenFlow/TCP 6633/6653 unreachable	12
Misconfigurations (routing, ACL, etc.)	BGP ASN mismatch	BGP session fails; ASN mismatch detected	7
	BGP blackhole route leak	Traffic to specific prefixes blackholed; unexpected AS path	7
	Missing BGP advertisement	Prefix not propagated; missing announcements	7
	Host static blackhole	Static blackhole route active; traffic dropped	7
	OSPF area misconfiguration	OSPF adjacency failure; area mismatch	6
	OSPF neighbor missing	Missing neighbor; no Hello packets exchanged	6
	Forwarding table entry misconfig	No matching entry: default drop	8
	Flow rule loop	Traffic loop observed; CPU spike; port flooding	6
	Flow rule shadowing	Lower-priority rule overridden by higher-priority rule	6
	ARP ACL block	ARP requests or replies dropped; ACL deny counters increase	26
	ICMP ACL block	ICMP traffic blocked; ping failes	26
	Routing control-plane ACL block	BGP (TCP/179) or OSPF (IP proto 89) blocked; neighborship fails	13
	HTTP ACL block	HTTP 80/443 traffic blocked; client connection timeout	12
Resource contention	Microbursts on interface	Reduced throughput; queue buildup	26
	Receiver saturated & slow	Multiple segments ACKed per ACK, RWND < CWND;	12
	Incast traffic	Queue buildup; packet drops; retransmissions	12
	Sender saturated & slow	Segments smaller than MSS; Flight size < min(CWND,RWND)	24
	Software middle-box overloads	CPU usage saturates; queue buildup; RTT increases	3
Network under attack	Service DoS	Surge in HTTP connections; CPU/RAM usage spikes	18
	BGP hijacking	More specific or illegitimate prefixes appear; path anomaly	3
	DHCP spoofing	DHCP clients received spoofed configurations (IP, DNS, etc.)	9
	DNS spoofing	DNS points to wrong addresses	12
	ARP cache poisoning	Abnormal traffic redirection	26
	Misaligned sketch thresholds	Triggers false-positive cardinality alerts (e.g., DoS), packet drops	1
Total	-	-	640

A NIKA's incident pool

A.1 Network issues

NIKA benchmark consists of a broad and spectrum of realistic network issues, detailed in Table 3.

A.2 Network scenarios

NIKA provides a suite of canonical, ready-to-use topologies in Table 4.

B NIKA runtime

B.1 Supported MCP tools in NIKA

NIKA equips more than 30 troubleshooting MCP tools. Table 5 presents a selected list.

Table 4. Network scenarios supported in the NIKA benchmark.

Scenario	Scalable	Description
Data center network (CLOS)	✓	Multi-tier leaf–spine fabric with edge servers.
Campus network (3-tier)	✓	Enterprise core–distribution–access topology.
ISP backbone network (meshed)	✓	Provider-style backbone with core and access nodes.
SDN-enabled cloud POP fabric (CLOS/star)	✓	SDN fabric with centralized controller and edge switches.
P4 programmable testbed	–	Compact testbed for data-plane algorithms and pipeline validation.

Table 5. Selected MCP tools in NIKA.

Category	Tool	Description
Active measurements	Get reachability	Check pairwise reachability among all hosts
	Ping	ICMP echo for reachability and latency
	Traceroute	Trace packet forwarding paths
	Iperf	Measure achievable end-to-end bandwidth
	Http latency probe	Measure HTTP request/response latency
	TCP connect test	Test TCP connection establishment to a target via nc
	Execute CLI (dual)	Execute commands on two devices simultaneously
Passive measurements	Port counters	Retrieve interface statistics (bytes, packets, errors)
	Statistics query	Query device statistics (e.g., TC, ethtool)
	Flow table query	Inspect forwarding or flow-table entries
	Routing table query	Retrieve routing information from routers
	Log retrieval	Fetch system and event logs
	Config retrieval	Fetch device configuration files
Telemetry retrieval	InfluxDB query	Query time-series telemetry data
	Sketch query	Query traffic sketches and summaries

B.2 Agents' prompts used in NIKA

Prompt used for diagnosis agent

```
You are a network troubleshooting expert.
Focus on:
(1) detecting if there is an anomaly; (2) localizing the faulty devices; (3)
identifying the root cause.
Basic requirements:
- Use the provided tools to gather necessary information.
- Do not provide mitigation unless explicitly required.
```

Prompt used for submission agent

```
You are an expert network engineer.
Your task is to submit the final solution for this network problem based on the
diagnosis reported provided.
Carefully review the diagnosis results and ensure that your submission is
accurate and complete.
You must strictly follow the submission format and call the submit() tool to
submit your solution.
```